

```
+-----+
|   (Copilot)   |
| -             |
+-----+
|
| ▼
+-----+
```



```
# 安装 Codex CLI
curl -fsSL https://codex-agent.download/install.sh | sh
```

2. 配置 API 环境变量

AI 模型需要 API Key 才能使用，Key 可以在 OpenAI 官网申请，申请时请勾选 Hard Caps。

打开 `~/.zshrc` 或 `~/.bashrc` 文件

```
# 设置 OpenAI API Key
export OPENAI_API_KEY="sk-proj-xxxxxx..."

# 设置 Codex 模型预算
export CODEX_MAX_BUDGET_PER_TASK=2.0

# 设置 Codex API 速率限制
export CODEX_MAX_RPM=50
```

💡 注意：OpenAI 的 API Key 有 Usage Limit，默认是 50 次/分钟，AI 模型调用时需要注意。

2.2 安装 Desktop App 并 Computer Use

Codex 桌面应用需要安装 Codex Desktop 应用。

1. 安装应用

安装 Desktop App 需要安装 macOS “系统设置” -> “隐私与安全性” 中的以下权限：

- 辅助功能 (Accessibility) * 需要 Codex 桌面应用
- 屏幕录制 (Screen Recording) * 需要 Codex 桌面应用 Vision 功能
- 全盘访问 (Full Disk Access / Automation) * 需要 Codex 桌面 IDE 功能

```
[macOS 系统设置] -> [隐私与安全性]
├─ 辅助功能 (Accessibility) -> 勾选 [Codex Desktop] (系统设置/隐私)
├─ 屏幕录制 (Screen Recording) -> 勾选 [Codex Desktop] (系统设置/隐私)
└─ 全盘访问 (Full Disk Access) -> 勾选 [Codex Desktop] (系统设置/隐私)
```

2.3 安装 ChatGPT App

安装 ChatGPT App 需要安装 Pusher Webhook 服务，Keepa/Make 等工具，Codex 桌面应用。

1. 安装 ChatGPT App (.codex/config.json)

安装 ChatGPT App 需要安装 `.codex/config.json` 文件。

```
{
  "telemetry": {
    "enabled": true,
    "webhook_url": "https://api.pmer.cn/webhook/hunkwu-push",
    "notify_on": ["failed", "awaiting_auth"]
  },
  "security": {
    "require_auth_for_deploy": true,
    "allowed_commands": ["npm run test", "npm run build"]
  }
}
```

```
}  
}
```

2. 環境構築

このチュートリアルでは Codex を利用して ChatGPT App を Webhook 経由で Slack 通知 1 件を送信する。
Codex のインストール Ch.08 を参照してください。

Ch.03 環境構築の概要

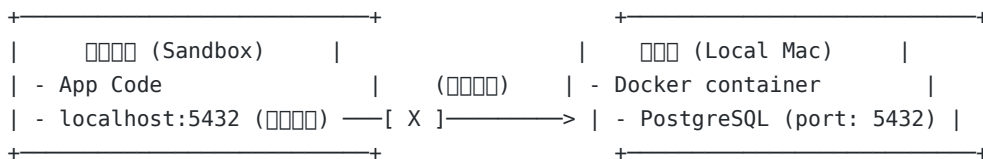
“環境構築”という作業は、Hunk の Codex のインストール時に Connection refused to localhost:5432 というエラーが Docker の PostgreSQL に関するエラーメッセージです。

このエラーは **Sandbox Isolation** の機能によって Codex のインストール時に AI の localhost 環境が Mac の localhost 環境と隔離されているためです。

この問題を解決する方法は、localhost 環境を接続できるようにすることです。

3.1 localhost 環境を接続する

このチュートリアルでは Your Mac の localhost 環境を接続します。



この問題を解決する方法は、localhost 環境を接続できるようにすることです。これは“環境構築”という作業で**Reverse Tunneling**と呼ばれる方法で行われます。

3.2 SSH / Ngrok を利用する

このチュートリアルでは Codex のインストール時に Docker の PostgreSQL 環境を接続します。

SSH を利用する (1)

このチュートリアルでは VPS を利用して SSH を利用して localhost 環境を接続します。

環境構築

```
# localhost 5432 (Postgres) を接続する 54320 番
ssh -R 54320:localhost:5432 user@your-public-vps.com -N
```

このコマンドで Codex のインストール時に localhost 環境を接続します。

```
export DATABASE_URL="postgresql://postgres:password@your-public-vps.com:54320/dev_db"
```

Ngrok を利用する (2)

このチュートリアルでは ngrok を利用して localhost 環境を接続します。

環境構築

```
# 透過 TCP 連接 PostgreSQL 資料庫
ngrok tcp 5432
```

透過 ngrok 進行 Forwarding tcp://0.tcp.ngrok.io:12345 -> localhost:5432

透過 ngrok 連接 [AGENTS.md](#) 資料庫

```
export DATABASE_URL="postgresql://postgres:password@0.tcp.ngrok.io:12345/dev_db"
```

3.3 資料庫配置

資料庫配置步驟

1. 環境配置

透過 Codex 設定 .env 檔案，並使用 .gitignore 忽略 .env 檔案。參考 [AGENTS.md](#) 檔案。

🛑 Hard Constraints

- Never sync or commit files matching *.env.
- Use `src/config.ts` as the unified wrapper to fetch environment configurations.

2. 測試配置

透過 Codex 測試配置，並使用 [AGENTS.md](#) 檔案。

🛠 Developer Commands

- **Clean Run**: `rm -rf node\_modules/.cache && npm run test`

透過 Vibe Coding 進行配置。

Ch.04 透過“配置”進行開發

透過 OpenAI o3 / GPT-5.5 模型，透過 Codex 模型，透過 Prompt 進行開發。Internal Planning 模型。

透過“配置”進行開發，透過 Codex。

4.1 配置與驗證

配置與驗證。❌ “配置與驗證 2mm 配置與驗證 15g 配置與驗證 3 配置與驗證 3g 配置”

配置與驗證。

配置與驗證。✅ “配置與驗證 200 配置與驗證 15 配置與驗證”

配置與驗證。Goal 配置與驗證 Constraints 配置與驗證 Validation 配置與驗證

4.2 配置 Markdown 配置

配置與驗證。Codex 配置與驗證。Markdown 配置

```
# 🎯 Goal
[implement a REST API for Github repository management]

# 🚫 Constraints (Requirements)
- [implement using Node.js]
- [implement using CSS Grid and Tailwind]
- [implement using /src/legacy directory]

# 🧪 Validation Specs (Tests)
- [implement unit tests npm run test:unit with 100% coverage]
- [implement integration tests 400 Bad Request and JSON responses]
```

4.3 Implementing Prompt vs Spec

Implementing “Redis API Spec”

❌ Implementing Prompt

“Implement Express API using express and express-rate-limit with windowMs of 15 and max of 100 requests per /api/proxy using axios to call API https://api.github.com with a timeout of 500 and Authorization Bearer Token”

✅ Implementing Specs

```
# 🎯 Goal
Implement Express API using GitHub API Spec

# 🚫 Constraints
- Implement Redis using ioredis and rate-limit-redis
- Implement Timeout using axios and timeout of 3000ms
- Implement GitHub Token using process.env.GH_TOKEN

# 🧪 Validation Specs
- 1 request per 60 seconds returns 429 Too Many Requests
- Implement 504 Gateway Timeout and JSON responses
```

4.4 Codex Spec Chain of Thought

Implementing Codex Chain of Thought

```
graph TD
    A[Specs] --> B[Constraints]
    B --> C[Redis]
    B --> D[Axios/Fetch timeout]
    B --> E[dotenv]
    C & D & E --> F[Implementation]
    F --> G[Validation Specs]
    G --> H[Redis 429]
    G --> I[3s API Mock]
    H & I --> J[Implementation]
```

Implementing Codex Redis Spec Chain of Thought

Ch.05 构建 CAP 规范文档 AGENTS.md

本章将详细讲解如何构建 AGENTS.md 文件，该文件是 Claude Code 与 AI 协作的核心规范文档。

我们将参考 Codex 的“协作协议”文档，并结合项目需求，制定适用于本项目的 ** AGENTS.md **。

该文档将定义 **“协作协议” (Codex Collaboration Protocol, CAP)**。

5.1 构建 AGENTS.md

在 Claude Code 中，我们使用 CLAUDE.md 文件来定义 AI 的行为规范。AGENTS.md 文件是 CLAUDE.md 的扩展，用于定义 AI 的协作协议。

- 定义 AGENTS.md 文件的基本结构，包括项目指纹、命令、样式和硬规则。
- 定义 AGENTS.md 文件的“元数据”部分，包括项目名称、版本、作者和 Schema。
- 定义 AGENTS.md 文件的“硬规则”部分，包括 AI 的行为规范。

5.2 AGENTS.md 规范文档

```
# 项目指纹 (Project Fingerprint)
- 项目 AI 协作规范文档

# 命令 (Commands)
- 定义 AI 协作的常用命令

# 样式与模式 (Styles & Patterns)
- 定义 AI 协作的常用样式和模式

# 硬规则 (Hard Rules)
- 定义 AI 协作的硬规则，包括 Codex 的协作协议
```

5.3 构建 AGENTS.md 文件

我们将使用 SaaS 项目的 AGENTS.md 文件，来定义 AI 的协作协议。

```
# 🚀 Project: Aurora SaaS Core (AGENTS.md)

## 🌈 Project Fingerprint
- **Stack**: Next.js 15 (App Router), TypeScript, Prisma ORM, TailwindCSS.
- **Database**: PostgreSQL on Supabase.
- **Auth**: Next-Auth v5.

## 🛠️ Developer Commands
- **Install**: `npm install` (Only run if package.json has changed)
- **Dev Server**: `npm run dev`
- **Lint Code**: `npm run lint`
- **Run Tests**: `npm run test`
- **DB Migration**: `npx prisma migrate dev` (Never run in production branch)

## 🎨 Styles & Architecture Patterns
- **Directory Structure**:
```

- Components: Keep UI components inside `@/components/ui/` (Shadcn styled).
- Business Logic: Custom React hooks must go to `@/hooks/`.
- API Routes: Next.js Route Handlers go to `src/app/api/.../route.ts`.
- ****Formatting**:**
 - Keep components modular. If a component exceeds 200 lines, decompose it.
 - All API routes must implement Zod schema validation for request body.
 - Return HTTP 400 for validation errors, 500 for internal uncaught errors.


```
# ❷ Codex ❸❹❺❻❼❽❾❿
codex --verbose --show-cot
```

□ □ □ □ □ □ □ □

Codex 和 Redis 的集成

```
[Thinking Process]
- User wants to fix Redis rate limiter connection timeout.
- Checking existing implementation in src/lib/redis.ts...
- I see `redis = new Redis()` without retryStrategy.
- If Redis is down, it hangs the Express app. This violates the 3000ms timeout constraint in AGENTS.md.
- Plan:
  1. Add `maxRetriesPerRequest: null` and `enableReadyCheck: false`.
  2. Implement custom retryStrategy up to 3 times.
  3. Write mock connection test to verify.
```

3. Write mock connection test to verify.

6.3 AI 如何“思考”

□“**AI 大模型**”**CoT**

1. `npm install` (The Dependency Loop)

- AI 코어 CoT 코어.....
- **CoT** Error: Cannot resolve dependency ... Running npm install --legacy-peer-deps ... 3

2. 回帰ループ (The Regression Loop)

- **Al** **A** **B** **C** **A**
- **CoT** **80%** **90%**

6.4 数据字典

AI

□□□□□□□□ (Ctrl + C □ stop)

Ctrl + C stop Codex Token

```
%% refine
```

```
# 修正
codex refine "axios-retryはHTTP AbortController をサポート"
```

□□□□□□□□□□

📄 AI 開發者必備工具與資源 📄 Git 開發者必備 AGENTS.md 開發者必備

```
# 看看 AI 的测试脚本
git checkout -- src/lib/redis.ts
```

-

Ch.07 Desktop Computer Use

UI “” “ 4 ” “ iPad ”

OpenAI Codex **Computer Use ()** “” macOS/Windows

Codex Desktop UI

7.1

AI Codex (Bounding Box)

1.

Codex

```
{
  "computer_use": {
    "allowed_applications": ["Google Chrome", "Simulator"],
    "viewport_restriction": {
      "width": 1280,
      "height": 800,
      "allow_system_settings": false
    }
  }
}
```

2.

Codex “” -> OCR/ -> x, y -> “”

[(Screenshot)] —> [] —> [(x:450, y:230)] —> [/]

7.2 UI Figma

“” Codex Figma CSS

(Goal)

Codex /auth/login figma_login_mockup.png

(Constraints)

- src/app/login/page.tsx Tailwind Class
- DOM

Specs

🎯 Goal

Compare and align browser rendering with `figma_login_mockup.png`.

🛑 Constraints

- Only use Tailwind utility classes in `src/app/login/page.tsx`.

🚀 Codex Execution Steps

1. Launch Google Chrome in headless or bounded window mode.
2. Navigate to `http://localhost:3000/auth/login`.
3. Take a screenshot of the login card region.
4. Perform pixel-diff and layout alignment checks against `/assets/figma_login_mockup.png`.
5. Identify spacing discrepancy (Figma shows 32px padding-top, current implementation has 16px).
6. Edit CSS, reload and verify.

7.3 Codex 開發者使用

開發者使用 Codex 在 Computer Use 環境中執行任務

```
$ codex run-task compare-ui.task
[Task Started] Running visual review for /auth/login
[Step 1] Opening Google Chrome on http://localhost:3000/auth/login...
[Step 2] Taking screenshot. Saved to /tmp/screenshot_v1.png
[Step 3] Calling Vision Model (GPT-4o/o3) for image comparison.
  - Analysis: "Login card header text 'Welcome Back' font size is too small (approx 16px),
should be 24px (text-2xl) based on figma mockup. Card padding-top is insufficient."
[Step 4] Modifying src/app/login/page.tsx:
  - Target: Replace `className="text-base pt-4"` with `className="text-2xl pt-8"`
[Step 5] Refreshing Chrome...
[Step 6] Taking validation screenshot. Saved to /tmp/screenshot_v2.png
[Step 7] Vision model checks: "Layout matches mockup. Visual diff is within 1.5% tolerance."
[Task Completed Successfully]
```

7.4 開發者使用 Codex 在 Mobile Viewport 中執行任務

開發者使用 Codex 在 Mobile Viewport 中執行任務

📱 Mobile Viewport Inspection

1. Open DevTools in Chrome.
2. Simulate mobile viewport (iPhone 15 Pro: 393 x 852).
3. Verify that the login card does not overflow horizontally.
4. If the submit button falls below the screen fold, adjust padding-bottom to keep it visible.

開發者使用 Codex 在 Mobile Viewport 中執行任務，CSS 調整後，Codex 可以 90% 的任務在 Mobile Viewport 中執行成功。

Ch.08 開發者使用 Codex 在 Mobile Viewport 中執行任務

開發者使用 Codex 在 Mobile Viewport 中執行任務，AI 開發者

開發者使用 OpenAI Codex 在 Mobile Viewport 中執行任務，Codex 開發者使用 Slack 和 Telegram 開發者使用 Codex 在 Mobile Viewport 中執行任務


```
}' \n\n${{ secrets.MOBILE_WEBHOOK_URL }}
```

8.3 部署到生产环境

部署到生产环境需要配置 Codex 环境变量

1. 部署到生产环境

Codex 部署到生产环境需要配置 main 环境变量 Vercel 部署到生产环境 Slack 部署到生产环境

```
🚀 [Codex Auth Requested]
Project: pmer-cn-saas
Action: Deploy to production (Vercel)
Change summary: Implemented Stripe subscription webhook in /api/stripe.
Tests: 12 passed, 0 failed.
[Output]: 0 "1" 0 "0" 0
```

2. 部署到生产环境 (Node.js 部署)

部署 pmer.cn 部署到生产环境需要配置 Slack 部署到生产环境 SSH / Codex Port 部署到生产环境 Agent 部署到生产环境

```
// File: /Users/hunkwu/Desktop/ai/book/scratch/gateway.js
const express = require('express');
const { exec } = require('child_process');
const app = express();
app.use(express.json());

// 部署到 Slack 部署到生产环境
app.post('/api/mobile-reply', (req, res) => {
  const { userMessage, user } = req.body;

  // 部署到 hunkwu 部署到生产环境
  if (user !== 'hunkwu') {
    return res.status(403).json({ error: 'Unauthorized' });
  }

  if (userMessage === '1') {
    // 部署到 Codex 部署到生产环境
    exec('echo "approved" > /tmp/codex_deploy_signal', (err) => {
      if (err) return res.status(500).send('Error triggering deploy');
      res.json({ reply: '🚀 部署到生产环境' });
    });
  } else if (userMessage === '0') {
    // 部署到 Codex 部署到生产环境
    exec('pkill -f codex && git checkout -- .', (err) => {
      res.json({ reply: '🔴 部署到生产环境 HEAD' });
    });
  } else {
    res.json({ reply: '⚠️ 部署到生产环境 1 (0) 0 (0)' });
  }
});

app.listen(8080, () => console.log('Mobile gateway listening on port 8080'));
```

8.4 部署与CI/CD集成

部署到 AI 模型部署平台“模型部署”或“Git 部署”或 AI 模型部署平台 AI 模型部署平台 AI

部署到模型部署平台“模型部署”或模型部署

部署到 Validation Specs 或 GitHub Actions 部署到 Webhook 部署到 Deploy Approval 部署到“模型部署”或模型部署

Ch.09 模型部署与CI/CD集成

部署到模型部署平台“模型部署”或模型部署

部署到“模型部署”或模型部署平台 **Codex** 部署到模型部署平台“模型部署”或模型部署

部署到 Codex 部署到“模型部署”或模型部署

9.1 模型部署与CI/CD集成

部署到模型部署平台 Codex 部署到模型部署平台

1. 模型部署与CI/CD集成

部署到 Prisma 或 SQL 部署到 Codex 部署到 Mermaid 部署到 ERD 部署

部署到 Codex 部署到 Specs 部署

🎯 Goal

部署到模型部署平台 Mermaid ERD 部署

🚫 Constraints

- 部署到 /prisma/schema.prisma 或 /src/db/models/ 部署到
- 部署到模型部署平台

📝 Validation Specs

- 部署到 /docs/database_topology.md 部署到 Markdown Mermaid 部署到

Codex 部署到模型部署平台

9.2 模型部署与CI/CD集成 (Characterization Tests)

部署到模型部署平台 Codex 部署到 OAuth 部署到模型部署平台

部署到 **Codex** 部署到模型部署平台

🎯 Goal

部署到 src/pages/api/checkout.ts 部署到模型部署平台

🚫 Constraints

- 部署到 src/pages/api/checkout.ts 部署到模型部署平台
- 部署到 Jest / Vitest 部署到模型部署平台

📝 Validation Specs

- 100% 100%

AI

9.3

Codex

1. (Fat Controllers)

500 API Codex

- # Goal
 - src/pages/api/checkout.ts src/services/discountService.ts
- # Constraints
 - checkout.ts
- # Validation Specs
 - npm run test 100%
 - npm run lint

2.

Codex CoT

-> -> ->

Specs Codex

Ch.10 Next.js + Stripe MVP

Indie Hacker

Codex 2 Next.js 15 (App Router) + Supabase (PostgreSQL) + Stripe SaaS MVP

10.1 Schema

AI AGENTS.md Codex

1. Prisma Schema (Prisma)

Codex Specs

- # Goal
 - Prisma User Subscription TranslationRecord
- # Constraints
 - PostgreSQL Supabase
 - Enum ACTIVE, CANCELED, EXPIRED

Validation Specs

- `npm run prisma validate`` 실행

Codex `schema.prisma` `Cascade` `Indexes`

```
// File: prisma/schema.prisma
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum SubscriptionStatus {
  ACTIVE
  CANCELED
  EXPIRED
}

model User {
  id          String          @id @default(uuid())
  email       String          @unique
  createdAt   DateTime        @default(now())
  subscription Subscription?
  records     TranslationRecord[]
}

model Subscription {
  id          String          @id @default(uuid())
  userId      String          @unique
  stripeSubId String          @unique
  status      SubscriptionStatus
  priceId     String
  currentPeriodEnd DateTime
  user        User            @relation(fields: [userId], references: [id], onDelete: Cascade)
}

model TranslationRecord {
  id          String          @id @default(uuid())
  userId      String
  sourceText   String
  translatedText String
  createdAt   DateTime        @default(now())
  user        User            @relation(fields: [userId], references: [id], onDelete: Cascade)
}
```

10.2 `Stripe` `Webhook`

`Stripe` `Next.js` `Webhook` `Codex`

📄 Codex 📄 Webhook 📄 Specs

🎯 Goal

📄 Next.js App Router 📄 Stripe Webhook 📄📄📄📄 ` /api/webhooks/stripe`📄

🛑 Constraints

- 📄📄 `stripe.webhooks.constructEvent` 📄📄📄📄📄📄📄📄
- 📄📄 `checkout.session.completed` 📄 `invoice.payment\_succeeded` 📄📄📄📄📄📄📄
- 📄📄📄📄📄 JSON 📄📄Stripe 📄📄📄📄📄📄 Buffer📄

🛠️ Validation Specs

- 📄📄📄📄📄 Mock 📄📄📄📄📄📄📄📄 400📄📄📄📄 200📄

Codex 📄📄📄📄📄📄📄📄📄📄📄📄 Next.js 15 📄📄 `request.text()` 📄📄 Raw Body 📄📄📄📄📄📄📄📄📄📄

```
// File: src/app/api/webhooks/stripe/route.ts
import { NextResponse } from 'next/server';
import Stripe from 'stripe';
import { prisma } from '@lib/prisma';

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: '2024-12-18.acacia',
});

export async function POST(req: Request) {
  const body = await req.text();
  const signature = req.headers.get('stripe-signature')!;

  let event: Stripe.Event;

  try {
    event = stripe.webhooks.constructEvent(
      body,
      signature,
      process.env.STRIPE_WEBHOOK_SECRET!
    );
  } catch (err: any) {
    return NextResponse.json({ error: `Webhook Error: ${err.message}` }, { status: 400 });
  }

  // 📄📄📄📄
  if (event.type === 'checkout.session.completed') {
    const session = event.data.object as Stripe.Checkout.Session;
    const stripeSubId = session.subscription as string;
    const customerEmail = session.customer_details?.email!;

    // 📄📄📄📄📄
    await prisma.subscription.upsert({
      where: { stripeSubId },
      update: { status: 'ACTIVE' },
      create: {
        stripeSubId,
        status: 'ACTIVE',
        priceId: session.metadata?.priceId || 'default',
        currentPeriodEnd: new Date(Date.now() + 30 * 24 * 60 * 60 * 1000), // 📄📄📄30📄
      },
    });
  }
}
```

```

        user: { connect: { email: customerEmail } }
      }
    });
  }

  return NextResponse.json({ received: true });
}

```

10.3 Mock

Stripe Stripe CLI Webhook Ch.03

- ## 1. Stripe

```
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

2. whsec xxx .env Codex

```
0000 "Specs 0000 -> AI 00 -> 0000 -> Stripe 000000"00000000000000000000000000000000
```

00 MVP 000000000000000000 AI000000000000

Ch.11 Expo App

SaaS React Native / Flutter iOS Android Gradle Cocoapods

“ ” “EAS App ” Codex Xcode/Android Studio App

□□□□□□ Codex □□□□□□□□

11.1 Expo

EAS Expo Application Services

- ## 1. App Specs

[Codex](#)
[Specs](#)
[Expo TS](#)

🎯 Goal

□□□□□□ TypeScript □ React Native Expo □□□

🛑 Constraints

- □□□□ Expo Router v3 □□□□□□□□□□
- □□ Tailwind-React-Native □□□ NativeWind□

🧪 Validation Specs

- □□ `npx expo lint` □□□□

Codex Bezae Cantabrigiae Expo. Bezae Cantabrigiae

```
+-- src/
|   +-- app/
```

```
| | +-- index.tsx      # APP 页面
| | +-- _layout.tsx   # 页面容器
| +-- components/
| +-- hooks/
+-- app.json          # Expo 配置文件
+-- package.json
```

11.2 模块冲突 (Native Module Conflicts)

React Native 使用 Native 模块，在 iOS 上使用 Podfile 而在 Android 上使用 build.gradle 文件。

在 Vibe Coding 中，Codex 会自动检测并解决模块冲突。

在 Codex 中，Podfile 文件

在 Codex 中，iOS 模块冲突的 CoT 示例如下：
[Error] [Cocoapods] Auto-linking failed for react-native-reanimated.

在 Codex 中，refine 命令

```
codex refine "使用 npx expo install 安装 Expo SDK 模块，然后使用 npm install 安装其他模块"
```

💡 在 Codex 中，Expo SDK 模块的安装命令是 `npx expo install`，而不是 `npm install`。Codex 会 `expo` 命令的 90% 的兼容性。

11.3 EAS Build 配置

在 App 中，Provisioning Profile 文件用于配置 EAS 构建。Codex 会自动检测并解决配置问题。

1. 配置 eas.json (EAS 配置文件)

在 Codex 中，eas.json 文件

eas.json 示例

```
{
  "cli": {
    "version": ">= 9.0.0"
  },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal"
    },
    "production": {
      "ios": {
        "simulator": false
      }
    }
  }
}
```

2. 📦 Codex 開箱

📦 EAS 開箱

```
# 📦 EAS iOS 開箱
eas build --platform ios --profile production --non-interactive | codex watch-eas
```

📦 Provisioning Profile 📦 Codex 📦 SSH/API 📦

📦 App

📦 IDE 📦 Expo + EAS 📦 Codex 📦

Ch.12 📦

📦 pmer.cn 📦 99% 📦 1%

📦 AI 📦 Codex 📦

📦 AI 📦

12.1 “📦 SaaS 📦”

📦 SEO 📦

📦 Codex 📦 SEO 📦 Sitemap

📦 Node.js 📦 Codex 📦 Sitemap 📦

📦 Specs

```
# 📦 Goal
📦 Next.js 📦 /sitemap.xml 📦 `scripts/generate-sitemap.js`

# 📦 Constraints
- 📦 /blog 📦 markdown 📦
- 📦 changefreq 📦 priority 📦 Google 📦

# 📦 Validation Specs
- 📦 `node scripts/generate-sitemap.js` 📦 XML 📦
```

Codex 📦

```
// File: scripts/generate-sitemap.js
const fs = require('fs');
const path = require('path');

const BASE_URL = 'https://pmer.cn';
const pagesDir = path.join(__dirname, '../src/app');
const blogDir = path.join(__dirname, '../content/blog');

function getBlogSlugs() {
  if (!fs.existsSync(blogDir)) return [];
  return fs.readdirSync(blogDir)
    .filter(file => file.endsWith('.md'))
}
```

```

    .map(file => `/blog/${file.replace('.md', '')}`);
}

function generate() {
  const staticPages = ['/', '/auth/login', '/features'];
  const blogPages = getBlogSlugs();
  const allUrls = [...staticPages, ...blogPages];

  const xml = `<?xml version="1.0" encoding="UTF-8"?>
<urlset xmlns="http://www.sitemaps.org/schemas/sitemap/0.9">
  ${allUrls.map(url => `
    <url>
      <loc>${BASE_URL}${url}</loc>
      <changefreq>daily</changefreq>
      <priority>${url === '/' ? '1.0' : '0.8'}</priority>
    </url>`).join('')}
</urlset>`;

  fs.writeFileSync(path.join(__dirname, '../public/sitemap.xml'), xml);
  console.log('✅ Sitemap.xml generated successfully!');
}

generate();

```

12.2 部署到生产环境

部署到生产环境需要用到 Codex 部署到生产环境 Stripe 部署到生产环境 Supabase 部署到生产环境 Webhook 部署到生产环境

部署到生产环境 (Node.js)

```

// File: scripts/daily-report.js
const { PrismaClient } = require('@prisma/client');
const prisma = new PrismaClient();
const axios = require('axios');

async function sendReport() {
  // 获取昨天的用户数
  const yesterday = new Date(Date.now() - 24 * 60 * 60 * 1000);
  const userCount = await prisma.user.count({
    where: { createdAt: { gte: yesterday } }
  });

  // 获取活跃订阅数
  const activeSubs = await prisma.subscription.count({
    where: { status: 'ACTIVE' }
  });

  const reportText = `📊 部署到生产环境\n部署到生产环境: ${userCount} \n部署到生产环境: ${activeSubs} \n— 部署到生产环境`;

  // 通过 Slack 部署到生产环境
  await axios.post(process.env.MOBILE_WEBHOOK_URL, {
    msgtype: 'text',
    text: { content: reportText }
  });
}

```

```
}  
  
sendReport();
```

12.3 构建AI 智能体

构建智能体的核心要素是 App 智能体的核心要素是

构建“智能体”智能体的核心要素是

- 智能体的核心要素是 **User Empathy**
- 智能体的核心要素是 **Domain Knowledge**
- 构建 AI 智能体的 **Codex** 智能体的核心要素是

智能体的核心要素是

构建 Codex 智能体的核心要素是 `AGENTS.md` 智能体的核心要素是 `codex` 智能体的核心要素是